Analysis of Information Diffusion on Threads: An Application of the SIR Model

Kevin Lin
Math 3440 Research Project

December 5, 2025

Abstract

The rapid spread of slang, memes, and viral content on social media platforms mirrors the biological transmission of infectious diseases. In this paper, I investigate how information spreads on the text-based platform Threads. By drawing an analogy between biological contagion and digital virality, I use the classical Susceptible–Infected–Recovered (SIR) model together with a custom Selenium-based web scraper to collect high-frequency time-series data on view counts for several viral posts within the Taiwanese community over one week. Fitting the resulting system of nonlinear ordinary differential equations to the observed data allows me to estimate key epidemiological parameters, including the transmission rate β, the recovery rate γ, and the basic reproduction number R_0. The results show that the SIR framework provides a reasonable approximation for algorithmic content spread and highlights the mathematical similarities between human communication and epidemic outbreaks.

1 Introduction

Social media platforms have transformed the way information spreads, allowing memes, slang, and news to spread through populations at a speed comparable to biological contagion. My interest in this phenomenon started during a conversation with Dr. Bio, who mentioned that he sometimes struggles to understand his son's slang because new terms appear and spread online so rapidly. This made me realize that the speed of language and meme diffusion on the internet resembles the spread of a pandemic, indicating that epidemic models could be a natural way to understand how new terms or memes propagate through social networks.

In this paper, I investigate the viral spread of posts on Threads, a text-based platform launched by Meta that relies heavily on algorithmic amplification to distribute content. Drawing a parallel between biological contagion and digital virality, I use the traditional Susceptible–Infected–Recovered (SIR) epidemic model to explain how users transition from a "susceptible" state (unaware of a post) to an "infected" state (viewing or sharing it) and eventually to a "recovered" state (losing interest or relevance) interest).

From the perspective of dynamical systems, a viral post can be seen as a trajectory in a phase space governed by a set of nonlinear ordinary differential equations (ODEs). A key question is the stability of the fixed points of the system: under what conditions does the "disease-free equilibrium" become unstable, leading to an outbreak of views? This transition is described by the basic reproduction number R_0. When R_0 exceeds the critical threshold $R_0 = 1$, the system experiences a qualitative change from minimal spread to rapid growth, which interpreted as a transcritical bifurcation.

To connect theory with data, I built a custom Selenium-based web scraper to track high-frequency time-series view counts of selected viral Threads posts within the Taiwanese community over one week. By fitting the SIR model to these data, I estimate key parameters such as the transmission rate eta and the basic reproduction number R_0, then evaluate how much algorithmic "pushing" of content behaves like biological transmission.

2 Data Collection and Methodology

To analyze the spread, I developed a Python-based web scraper to collect real-time view count data.

2.1 Data Source

I focused on the Taiwanese Threads community because it is highly active and closely aligned with the current trend toward short-form, reel-style content. According to the 2025 Reuters Institute Digital News Report, summarized in the *Taipei Times*, 46% of Taiwanese respondents use YouTube as a news source, and other social media platforms such as Line, Instagram, TikTok, and Threads have also increased as channels for news consumption. In this highly digital media environment, I selected five Threads posts that (i) were written in Chinese, (ii) showed early signs of rapid growth (more than [X] views in the first hour), and (iii) covered different topics such as travel, news, and everyday humor. For each post, I recorded the total view count at set time intervals over the course of one week, starting within the first hour after the post was posted.

2.2 Data Collection Procedure

I used ChatGPT as a coding assistant to help me design a custom Python script built on the Selenium framework. The script logs into my Threads account, navigates to each target post, and reads the view count directly from the relevant HTML elements on the page. I scheduled it to run automatically on a remote machine, recording the timestamp and cumulative view count for each post every 10 minutes for 7 consecutive days. Because Threads dynamically updates its interface, the scraper includes short delays to allow each page to finish loading, as well as simple error-handling rules that skip and log occasional failed requests. All data collected were stored in CSV format for subsequent analysis in Python and Mathematica.

2.3 Data Preprocessing

Before fitting the SIR model, I cleaned and standardized the raw data. First, I removed duplicate entries and clearly corrupted records, such as cases where the scraper captured a blank view count or repeated timestamps. Next, for each post, I aligned the time axis by setting $t = 0$ to be the first timestamp at which the post had a nonzero view count, and I computed a variable measuring the elapsed time in hours. To reduce small irregular jumps caused by interface refreshes and scraping delays, I smoothed the cumulative view counts using a simple three-point moving average. All subsequent modeling treats time in hours, so the fitted parameters β and γ are interpreted as per-hour transmission and recovery rates.

3 Mathematical Model

3.1 Classical SIR framework

The Susceptible–Infected–Recovered (SIR) model, originally formulated by Kermack and McKendrick in 1927, describes the transmission of an infectious disease through a fixed population of size N. The population is divided into three compartments:

- $S(t)$: number of susceptible individuals,
- $I(t)$: number of infected individuals,
- $R(t)$: number of recovered or removed individuals.

The dynamics is governed by the nonlinear ODE system

$$\begin{aligned} \frac{dS}{dt} &= -\frac{\beta SI}{N}, \\ \frac{dI}{dt} &= \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, \end{aligned} \tag{1}$$

where $\beta > 0$ is the transmission rate (contact) and $\gamma > 0$ is the recovery rate.

3.2 Interpretation for social media and basic reproduction number

To model information diffusion on Threads, we interpret the compartments as follows:

- **Susceptible (S):** Users in the algorithmic "feed pool" who have not yet seen the post but could be shown it by the recommendation system.
- **Infected (I):** Users who are currently viewing, liking, or sharing the post. These users act as "spreaders" by boosting the post's score.
- **Recovered (R):** Users who have scrolled past or interacted with the post and moved on are considered recovered; they are effectively immune to being counted as a new view.

In epidemiology, the basic reproduction number is defined by

$$R_0 = \frac{\beta}{\gamma}.$$

Early-time behaviour near the disease-free state

The I–equation in (1) is

$$\frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I.$$

When a Threads post is first published, almost nobody has seen it yet, so

$$I \approx 0 \quad \text{(almost no "infected" users)}, \qquad S \approx N \quad \text{(almost everyone is still susceptible)}.$$

In this very early stage we can treat $S(t)$ as approximately constant and replace it by N in the right-hand side. Then

$$\frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I \approx \frac{\beta NI}{N} - \gamma I = (\beta - \gamma)I = \gamma(R_0 - 1)I,$$

where $R_0 = \beta/\gamma$ is the basic reproduction number.

To see the implication of this approximation, we solve the linear differential equation

$$\frac{dI}{dt} = \gamma(R_0 - 1)\, I.$$

Separating variables gives

$$\frac{1}{I}\frac{dI}{dt} = \gamma(R_0 - 1) \quad \Longrightarrow \quad \frac{dI}{I} = \gamma(R_0 - 1)\, dt.$$

Integrating both sides with respect to t,

$$\int \frac{dI}{I} = \int \gamma(R_0 - 1)\, dt \quad \Longrightarrow \quad \ln|I(t)| = \gamma(R_0 - 1)t + C,$$

for some constant C. Exponentiating,

$$|I(t)| = e^{\gamma(R_0-1)t+C} = e^C e^{\gamma(R_0-1)t}.$$

Writing $A = e^C > 0$ and dropping the absolute value (since $I(t) \geq 0$ in our setting), we obtain

$$I(t) = A\, e^{\gamma(R_0-1)t}.$$

Using the initial condition at $t = 0$ gives $I(0) = A$, so the solution can be written as

$$I(t) \approx I(0)\, e^{\gamma(R_0-1)t}.$$

Thus, if $R_0 > 1$, the number of actively engaged users grows approximately exponentially at the beginning; if $R_0 < 1$, $I(t)$ decays back to zero. In everyday terms, R_0 tells us whether a small initial group of viewers will start a viral chain reaction ($R_0 > 1$) or whether the post fizzles out quickly ($R_0 < 1$). In this sense $R_0 = 1$ plays the role of a viral threshold.

On social media we usually do not observe $I(t)$ directly. Instead, we see the cumulative number of users who have ever seen the post. If $V(t)$ denotes this cumulative view count, then

$$V(t) = I(t) + R(t) = N - S(t),$$

because $S(t) + I(t) + R(t) = N$. Therefore, fitting the SIR model to the data is equivalent to choosing parameters so that the model quantity $N - S(t)$ tracks the observed cumulative views $V(t)$ as closely as possible.

3.3 Reduction to a two-dimensional system

For dynamical-systems analysis it is convenient to work with fractions rather than raw counts. Thus, we efine

$$s(t) = \frac{S(t)}{N}, \qquad i(t) = \frac{I(t)}{N}, \qquad r(t) = \frac{R(t)}{N},$$

so that

$$s(t) + i(t) + r(t) = 1 \quad \text{for all } t.$$

In these variables, (1) becomes

$$\begin{aligned} \frac{ds}{dt} &= -\beta si, \\ \frac{di}{dt} &= \beta si - \gamma i, \\ \frac{dr}{dt} &= \gamma i. \end{aligned} \tag{2}$$

The variables now represent fractions of the effective audience. Since $r(t) = 1 - s(t) - i(t)$, we can eliminate r and obtain a two-dimensional system for (s, i):

$$\begin{aligned} \frac{ds}{dt} &= -\beta si, \\ \frac{di}{dt} &= \beta si - \gamma i. \end{aligned} \tag{3}$$

The admissible region is

$$\Omega = \{(s, i) \in \mathbb{R}^2 : s \geq 0,\ i \geq 0,\ s + i \leq 1\},$$

which corresponds to nonnegative fractions with total at most one. One can check that Ω is forward invariant: solutions that start in Ω remain in Ω for all $t \geq 0$.

3.4 Fixed points, Jacobian matrix, and eigenvalues

Fixed points (s^*, i^*) of (3) satisfy

$$\frac{ds}{dt} = -\beta s^* i^* = 0, \qquad \frac{di}{dt} = i^*(\beta s^* - \gamma) = 0.$$

Hence either $i^* = 0$ or $s^* = 0$, and either $i^* = 0$ or $s^* = \gamma/\beta$. Within the region Ω the only fixed points are the *disease-free* (or "view-free") states

$$(s^*, i^*) = (s^*, 0), \qquad 0 \leq s^* \leq 1,$$

which form a line along the s-axis. In particular, $(1, 0)$ corresponds to the initial state where none of the audience has seen the post.

To study local behaviour near a fixed point we linearise (3). The Jacobian matrix of the vector field is

$$J(s, i) = \begin{pmatrix} \dfrac{\partial}{\partial s}(-\beta si) & \dfrac{\partial}{\partial i}(-\beta si) \\ \dfrac{\partial}{\partial s}(\beta si - \gamma i) & \dfrac{\partial}{\partial i}(\beta si - \gamma i) \end{pmatrix} = \begin{pmatrix} -\beta i & -\beta s \\ \beta i & \beta s - \gamma \end{pmatrix}.$$

For a disease-free fixed point $(s^*, 0)$ this reduces to

$$J(s^*, 0) = \begin{pmatrix} 0 & -\beta s^* \\ 0 & \beta s^* - \gamma \end{pmatrix}.$$

The characteristic polynomial is

$$\det(\lambda I - J(s^*, 0)) = \lambda^2 + (\gamma - \beta s^*)\lambda = 0,$$

so the eigenvalues are

$$\lambda_1 = 0, \qquad \lambda_2 = \beta s^* - \gamma. \tag{4}$$

Using $R_0 = \beta/\gamma$, we can write

$$\lambda_2 = \beta s^* - \gamma = \gamma(R_0 s^* - 1). \tag{5}$$

Since $\gamma > 0$, the sign of λ_2 is the same as the sign of $R_0 s^* - 1$.

For later use it is convenient to record the corresponding eigenvectors. Let $v = (v_1, v_2)^\mathsf{T}$.

Eigenvector for $\lambda_1 = 0$. We solve

$$J(s^*, 0)v = 0.$$

From the first row we obtain

$$-\beta s^* v_2 = 0,$$

so for any fixed point with $s^* > 0$ we must have $v_2 = 0$, while v_1 is free. Thus we may choose

$$v^{(1)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

This eigenvector points along the s-axis and reflects the fact that the entire line $\{(s, 0) : 0 \le s \le 1\}$ consists of fixed points.

Eigenvector for $\lambda_2 = \beta s^* - \gamma$. Now we solve

$$\big(J(s^*, 0) - \lambda_2 I\big)v = 0.$$

Using (4), we have

$$J(s^*, 0) - \lambda_2 I = \begin{pmatrix} -\lambda_2 & -\beta s^* \\ 0 & 0 \end{pmatrix},$$

so the first row gives

$$-\lambda_2 v_1 - \beta s^* v_2 = 0 \quad \Longleftrightarrow \quad \lambda_2 v_1 + \beta s^* v_2 = 0.$$

Assuming $s^* > 0$, we can solve for v_2 in terms of v_1:

$$v_2 = -\frac{\lambda_2}{\beta s^*} v_1.$$

Taking $v_1 = 1$ yields the eigenvector

$$v^{(2)} = \begin{pmatrix} 1 \\ -\dfrac{\lambda_2}{\beta s^*} \end{pmatrix} = \begin{pmatrix} 1 \\ -\dfrac{\beta s^* - \gamma}{\beta s^*} \end{pmatrix}.$$

In particular, at the initial state $(s^*, i^*) = (1, 0)$ we have $\lambda_2 = \beta - \gamma$ and

$$v^{(2)}\big|_{(1,0)} = \begin{pmatrix} 1 \\ -\dfrac{\beta - \gamma}{\beta} \end{pmatrix},$$

which is precisely the unstable direction along which small perturbations grow when $R_0 > 1$.

3.5 Stability cases for $0 \le s^* \le 1$

From (5) the nonzero eigenvalue at a disease–free fixed point $(s^*, 0)$ is

$$\lambda_2 = \gamma(R_0 s^* - 1),$$

where $\gamma > 0$ and $R_0 > 0$. Because γ is positive, the sign of λ_2 is completely determined by the sign of the factor

$$R_0 s^* - 1.$$

We now examine the different possibilities for s^* in the interval $[0, 1]$.

Case 1: $0 < s^* < 1$**.** In this case we have a one–parameter family of fixed points $(s^*, 0)$ with s^* strictly between 0 and 1. For such an s^* we consider the quantity $R_0 s^* - 1$.

- If $R_0 s^* < 1$, then
$$R_0 s^* - 1 < 0 \quad \Longrightarrow \quad \gamma(R_0 s^* - 1) < 0 \quad \Longrightarrow \quad \lambda_2 < 0.$$
Thus the fixed point $(s^*, 0)$ has one zero eigenvalue and one negative eigenvalue, so it is stable in the infected direction: a small perturbation with $i(0) > 0$ decays back to $i = 0$.

- If $R_0 s^* > 1$, then
$$R_0 s^* - 1 > 0 \quad \Longrightarrow \quad \gamma(R_0 s^* - 1) > 0 \quad \Longrightarrow \quad \lambda_2 > 0.$$
In this case $(s^*, 0)$ is unstable in the infected direction: a small introduction of infected users grows away from $i = 0$.

Case 2: $s^* = 1$**.** This fixed point corresponds to the initial state $(s, i, r) = (1, 0, 0)$, where no user has seen the post. Substituting $s^* = 1$ into $\lambda_2 = \gamma(R_0 s^* - 1)$ gives

$$\lambda_2 = \gamma(R_0 \cdot 1 - 1) = \gamma(R_0 - 1).$$

- If $R_0 < 1$, then $R_0 - 1 < 0$, so
$$\lambda_2 = \gamma(R_0 - 1) < 0.$$
The disease–free fixed point $(1, 0)$ is stable: even if a few users start viewing the post, their activity dies out.

- If $R_0 = 1$, then
$$\lambda_2 = \gamma(R_0 - 1) = 0.$$
Here the fixed point is marginal (neither strictly stable nor unstable); this is the critical threshold.

- If $R_0 > 1$, then $R_0 - 1 > 0$, so
$$\lambda_2 = \gamma(R_0 - 1) > 0.$$
The fixed point $(1, 0)$ becomes unstable: a tiny initial group of infected users leads to growth in $i(t)$, corresponding to the onset of a viral cascade.

Case 3: $s^* = 0$**.** This fixed point corresponds to $(s, i, r) = (0, 0, 1)$, where everybody in the effective audience has already seen the post and moved on. Setting $s^* = 0$ in (5) yields

$$\lambda_2 = \gamma(R_0 \cdot 0 - 1) = \gamma(-1) = -\gamma < 0,$$

since $\gamma > 0$. Thus this fixed point always has one zero eigenvalue and one negative eigenvalue and is therefore stable: once the entire audience has recovered, small perturbations in i decay back to zero and the post cannot "come back to life" in this simple SIR model.

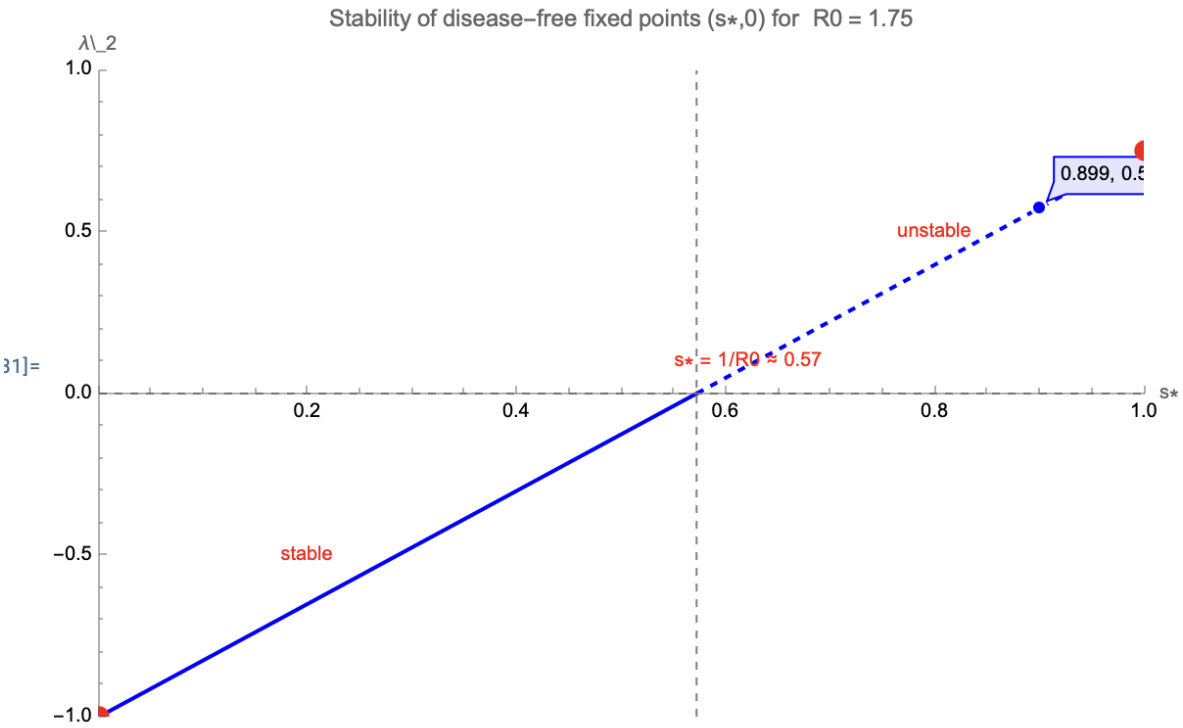


Figure 1: Stability of disease–free fixed points $(s^*, 0)$ for $R_0 = 1.75$.

Figure 1 summarizes the three cases above in a single diagram. For a fixed reproduction number $R_0 > 1$ we plot the nonzero eigenvalue $\lambda_2 = \gamma(R_0 s^* - 1)$ as a function of the susceptible fraction s^*. The line is drawn solid where $\lambda_2 < 0$ (stable in the infected direction) and dashed where $\lambda_2 > 0$ (unstable). The crossing at $s^* = 1/R_0$ corresponds to the condition $R_0 s^* = 1$ in Case 1. The end points $s^* = 0$ and $s^* = 1$ illustrate Cases 3 and 2 respectively.

3.6 Phase-plane structure and nullclines

Recall the two-dimensional SIR system in (s, i):

$$\frac{ds}{dt} = -\beta si, \qquad \frac{di}{dt} = \beta si - \gamma i. \tag{6}$$

Nullclines

The nullclines are obtained by setting the right-hand sides of (6) equal to zero.

For the s–equation,

$$\frac{ds}{dt} = -\beta si = 0 \quad \Longleftrightarrow \quad s = 0 \text{ or } i = 0.$$

For the i–equation,

$$\frac{di}{dt} = i(\beta s - \gamma) = 0 \quad \Longleftrightarrow \quad i = 0 \text{ or } s = \frac{\gamma}{\beta}.$$

Thus we have four nullcline pieces in the (s, i)–plane:

$$s = 0, \qquad i = 0, \qquad s = \frac{\gamma}{\beta}.$$

The line $i = 0$ is exactly the set of disease–free fixed points $(s^*, 0)$ found earlier.

Direction of motion in different regions

We now examine the sign of the derivatives to understand how trajectories move between these nullclines.

- For $i > 0$ and $s > 0$ we have
$$\frac{ds}{dt} = -\beta si < 0,$$
since $\beta > 0$, $s > 0$, and $i > 0$. This means that whenever there is a positive fraction of infected users, the susceptible fraction is always decreasing: trajectories move strictly leftward in the phase plane, and the susceptible pool is monotonically depleted over time.

- For $i > 0$, the sign of $\dfrac{di}{dt}$ is determined by the factor $(\beta s - \gamma)$:
$$\frac{di}{dt} = i(\beta s - \gamma).$$
Because $i > 0$, we have:
$$\begin{cases} \beta s - \gamma > 0 \iff s > \dfrac{\gamma}{\beta} \quad \Rightarrow \quad \dfrac{di}{dt} > 0, \\ \beta s - \gamma < 0 \iff s < \dfrac{\gamma}{\beta} \quad \Rightarrow \quad \dfrac{di}{dt} < 0. \end{cases}$$
Thus, above the vertical line $s = \gamma/\beta$ the infected fraction $i(t)$ increases, while below this line $i(t)$ decreases.

In words: as long as the susceptible fraction s is larger than the threshold $\gamma/\beta = 1/R_0$, the infection can still grow ($di/dt > 0$); once s falls below this threshold, the infection starts to die out ($di/dt < 0$).

Phase curves via $\dfrac{di}{ds}$

We can obtain an explicit family of phase curves by dividing the two equations in (6). For $s > 0$ and $i > 0$ we write i as a function of s and compute
$$\frac{di}{ds} = \frac{di/dt}{ds/dt} = \frac{\beta si - \gamma i}{-\beta si}.$$
Factor i in the numerator:
$$\frac{di}{ds} = \frac{i(\beta s - \gamma)}{-\beta si} = \frac{\beta s - \gamma}{-\beta s}.$$
Now split the fraction:
$$\frac{\beta s - \gamma}{-\beta s} = -\frac{\beta s}{\beta s} + \frac{\gamma}{\beta s} = -1 + \frac{\gamma}{\beta s}.$$
Hence
$$\frac{di}{ds} = -1 + \frac{\gamma}{\beta s}. \tag{7}$$

We now solve this first-order ODE for i as a function of s. Treat i as $i(s)$ and integrate both sides with respect to s:
$$\int \frac{di}{ds}\, ds = \int \left(-1 + \frac{\gamma}{\beta s}\right) ds.$$
The left-hand side is simply $i(s)$ (up to a constant), while the right-hand side integrates term-by-term:
$$i(s) = \int -1\, ds + \int \frac{\gamma}{\beta s}\, ds = -s + \frac{\gamma}{\beta} \ln s + C,$$
where C is a constant of integration.

Rearranging gives the implicit form of the phase curves:
$$i(s) + s - \frac{\gamma}{\beta} \ln s = C. \tag{8}$$
For each initial condition (s_0, i_0) with $s_0 > 0$ and $i_0 > 0$, we obtain a unique value of C:
$$C = i_0 + s_0 - \frac{\gamma}{\beta} \ln s_0,$$
and the entire trajectory of the SIR system through (s_0, i_0) lies on the corresponding level curve (8).

Geometrically, the nullclines $i = 0$ and $s = \gamma/\beta$ partition the phase plane into regions where the arrows point up/down and left. The phase curves (8) then describe how the susceptible fraction $s(t)$ and infected fraction $i(t)$ co-evolve as the system moves monotonically leftward until it eventually approaches the $disease$–free line $i = 0$.

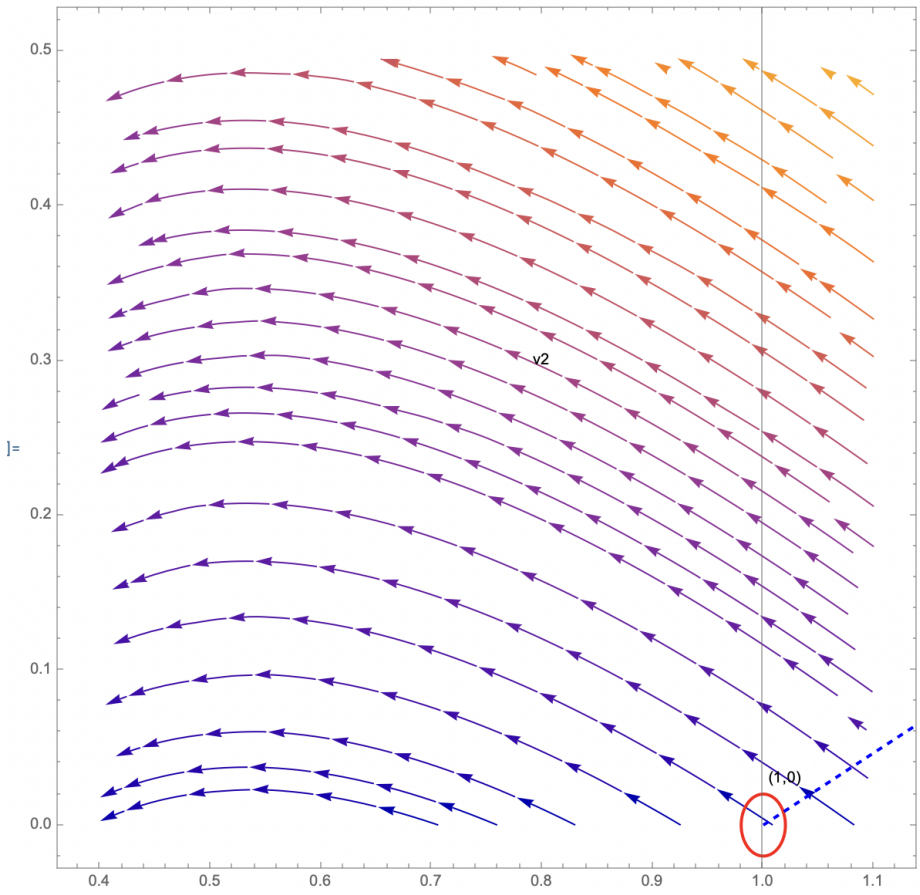


Figure 2: Phase portrait of the reduced SIR system in the (s, i)–plane for a representative value of R_0. The arrows show the direction field, the vertical line is the nullcline $s = \gamma/\beta$, and the red circle marks the disease–free fixed point $(1, 0)$.

4 Fitting the SIR model to Threads data

4.1 Overview of SIR fits for multiple posts

Figure 3 shows how well the classical SIR model can track how different Threads posts grow over time. The horizontal axis is the number of hours since we first started recording the post, and the vertical axis is the total view count. Each colour corresponds to one post. The dots are the actual data from Threads, while the smooth curves are the SIR model predictions using the parameters we fitted.

For each post we choose β and γ using nonlinear least squares so that the model quantity

$$V(t) = N - S(t)$$

matches the cumulative views as closely as possible. From these fitted parameters we compute

$$R_0 = \frac{\beta}{\gamma},$$

which now plays the role of a "viral reproduction number": it tells us how many new viewers one active viewer will on average generate at the start.

Several patterns are clear from the figure. Posts with R_0 much larger than 1. For example, the news or clothing posts in the legend show a sharp rise in views followed by a gradual saturation, giving the familiar S-shaped curve from epidemic models. Posts with R_0 close to 0 are almost flat lines: they never really take off and stay non-viral.

However, the Costco post stands out because its data points lie very close to the fitted SIR curve; visually, it is one of the best fits in the sample. This makes it a natural example for a more detailed phase–plane analysis.

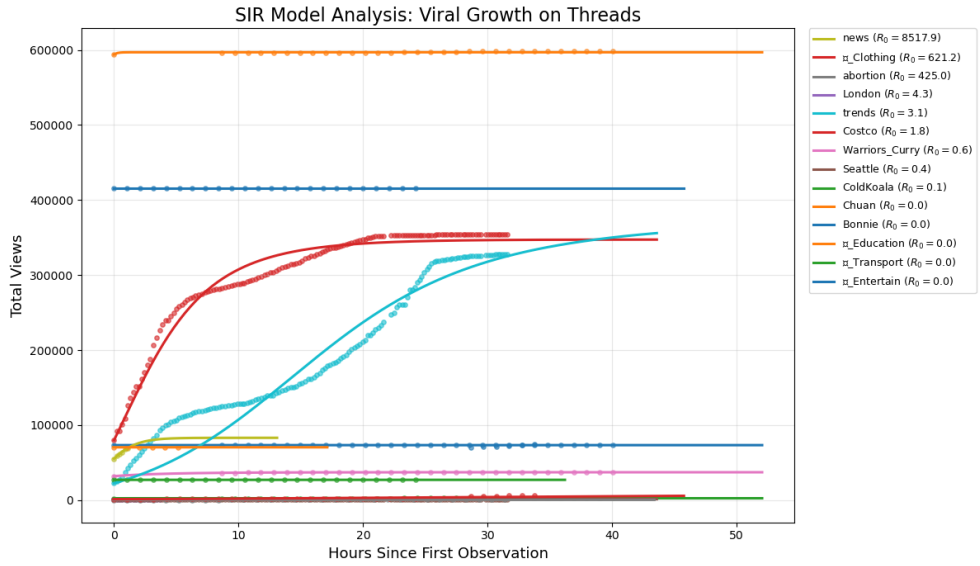


Figure 3: SIR model fits for several Threads posts. Dots show the observed cumulative views over time, and the solid curves are the fitted SIR trajectories $V(t) = N - S(t)$. The legend lists the post label together with its estimated basic reproduction number R_0.

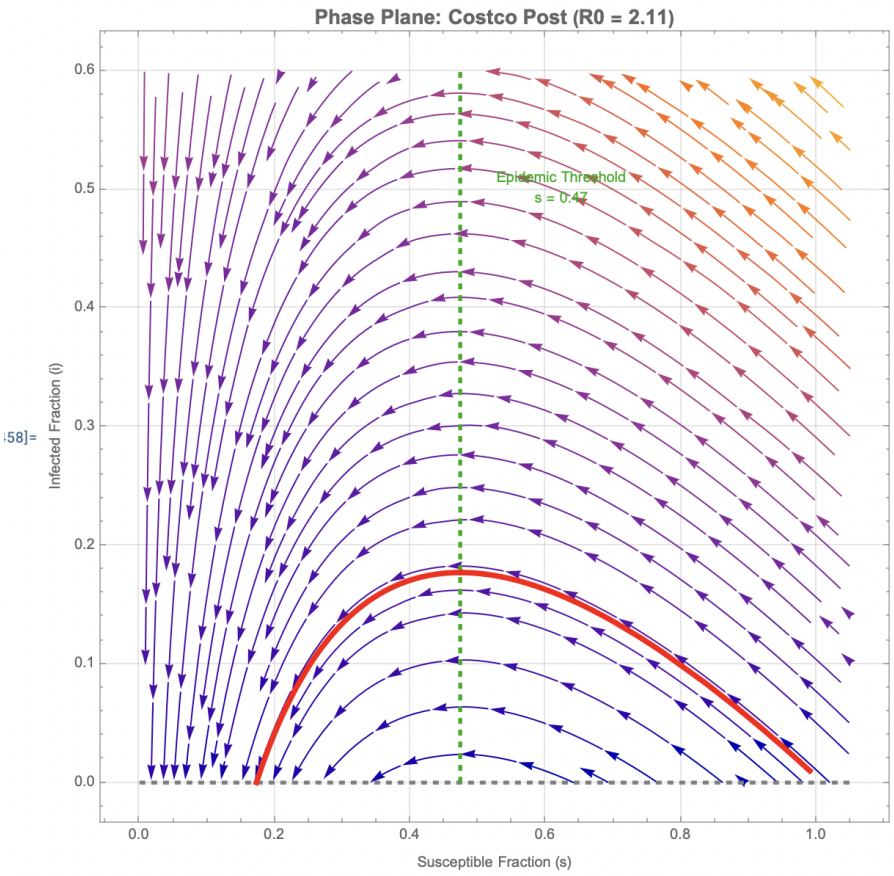


Figure 4: Phase portrait of the reduced SIR system in the (s, i)–plane for the Costco post, with fitted parameters $\beta = 0.38$ and $\gamma = 0.18$ (so $R_0 \approx 2.11$). The arrows show the direction field, the dashed vertical line is the nullcline $s = \gamma/\beta$, and the red curve is the trajectory starting from the initial state of the post.

4.2 Phase-plane interpretation for the Costco post

For the Costco post, we first used Python to estimate the parameters β and γ from the view data, as described above. We then plugged these values into the reduced SIR system and used Mathematica to draw the phase

portrait in the (s, i)–plane shown in Figure 4.

Here $s(t)$ is the fraction of the effective audience that is still susceptible (people who could see the post), and $i(t)$ is the fraction that is currently "infected" (people actively viewing and spreading it). Each arrow in the plot shows how (s, i) would move if The system was at that point.

The green dashed line marks the vertical nullcline

$$s = \frac{\gamma}{\beta} \approx 0.47.$$

To the right of this line, we have $s > \gamma/\beta$, so the model gives $\frac{di}{dt} > 0$ and the infected fraction grows. To the left, we have $s < \gamma/\beta$, so $\frac{di}{dt} < 0$ and the number of active spreaders begins to fall.

The red curve is the actual SIR trajectory for the Costco post, starting from a state with a large susceptible pool and a very small infected fraction. At first, the curve moves up and to the left: more users see and share the post while the pool of untouched viewers shrinks. Once the trajectory crosses the green line, the arrows tilt downward, and the red curve turns over. From that point on, even though there are still potential viewers left, the number of active spreaders is decreasing, and the post is slowly dying out.

All trajectories with $i > 0$ drift leftward and eventually approach the $disease$–free line $i = 0$. In particular, when $R_0 > 1$ the fixed point $(1, 0)$ is unstable in the infected direction: a tiny bump in i near $(1, 0)$ pushes the system away from that point and into the interior of the phase plane, which corresponds to a burst of viral growth. Later, once s falls below γ/β, the flow guides the system back down to $i = 0$, representing the end of the Costco post's life cycle in the Threads feed.

5 Conclusion

In this project, I showed that the classical SIR model from mathematical epidemiology can also describe how information spreads on the social media platform *Threads*. By building a Selenium-based scraper and collecting high-frequency view-count data for several Taiwanese Threads posts, I was able to fit the nonlinear SIR system to real time-series data and estimate the transmission rate β, the recovery rate γ, and the basic reproduction number R_0 for each post. Posts with $R_0 \gg 1$ displayed the familiar epidemic pattern of rapid initial growth followed by saturation, while posts with R_0 close to zero never became viral. In particular, for the Costco post the fitted parameters $\beta = 0.38$, $\gamma = 0.18$, and $R_0 \approx 2.11$ produced a trajectory that closely matched the observed view counts over time.

From the point of view of dynamical systems, the SIR model also provided a clear geometric picture of virality. Linearizing the reduced system in the (s, i)-plane showed how the basic reproduction number controls the stability of the disease-free line $i = 0$ and how the nullcline $s = \gamma/\beta$ acts as a threshold between growth and decay. For the Costco post, the phase portrait illustrated how a small initial "infection" near $(1, 0)$ is pushed into the interior of the phase plane when $R_0 > 1$, leading to a burst of viral growth, and how the trajectory eventually returns to $i = 0$ once the susceptible fraction falls below the threshold. This picture connects the abstract eigenvalue calculations from class to a concrete example of content spreading through an online audience.

At the same time, this approach has important limitations. The SIR model assumes a well-mixed population, constant parameters β and γ, and a fixed effective audience size, whereas real social-media platforms involve algorithmic boosting, network structure, and time-varying user behavior. My data set was also small and restricted to a single community and a short time window. Future work could extend this project by allowing $\beta(t)$ to change over time, embedding the SIR dynamics on a network, or combining mechanistic ODE models with machine-learning techniques to better capture algorithm effects and user heterogeneity. Even with these simplifications, the project demonstrates how tools from Math 3440—nonlinear ODEs, stability analysis, and phase portraits—can give insight into modern problems such as meme and information diffusion on social media.

6 Use of AI

I used ChatGPT to clarify concepts I did not understand in the paper, to check the grammar and clarity of my final paper, and to get help with LaTeX and coding.

References

[1] Huang, Chuangxia, Jie Cao, Fenghua Wen, and Xiaoguang Yang, "Stability Analysis of SIR Model with Distributed Delay on Complex Networks." *PLOS ONE* **11** (2016), e0158813.

[2] Jin, Fang, Wei Wang, Lingfei Zhao, Edward Dougherty, Yang Cao, Chang–Tien Lu, and Naren Ramakrishnan, "Epidemiological Modeling of News and Rumors on Twitter." In *Proceedings of the 7th Workshop on Social Network Mining and Analysis (SNA–KDD'13)*, ACM, 2013.

[3] Keeling, Matt J., and Pejman Rohani, *Modeling Infectious Diseases in Humans and Animals.* Princeton University Press, 2008.

[4] Kermack, W. O., and A. G. McKendrick, "A Contribution to the Mathematical Theory of Epidemics." *Proceedings of the Royal Society of London A* **115** (1927), 700–721.

[5] "Nearly half of Taiwanese use YouTube for news: report." *Taipei Times*, June 25, 2025. Available at `https://www.taipeitimes.com/News/taiwan/archives/2025/06/25/2003839237`.

[6] Pastor–Satorras, Romualdo, Claudio Castellano, Piet Van Mieghem, and Alessandro Vespignani, "Epidemic Processes in Complex Networks." *Reviews of Modern Physics* **87** (2015), 925–979.

[7] Strogatz, Steven H., *Nonlinear Dynamics and Chaos with Applications to Physics, Biology, Chemistry, and Engineering.* Chapman and Hall/CRC Press, 3rd ed., 2024.

[8] Zhao, Liang, Huidong Cui, Xiaoyan Qiu, Xin Wang, and Jürgen Wang, "SIR Rumor Spreading Model in the New Media Age." *Physica A: Statistical Mechanics and its Applications* **392** (2013), 995–1003.

A Python code for Normalising Data

Listing 1: Python script for normalizing Data

```
import os
import glob
import pandas as pd

# Paths
desktop_path = os.path.expanduser("/Users/kevinlin/Desktop")
master_path = os.path.join(desktop_path, "threads_MASTER_dataset.csv")
sir_ready_path = os.path.join(desktop_path, "threads_SIR_ready.csv")


def clean_views(x):
    x = str(x).replace(",", "")
    if "K" in x:
        return float(x.replace("K", "")) * 1000
    if "M" in x:
        return float(x.replace("M", "")) * 1000000
    return float(x)


def merge_to_master():
    """Merge all threads_*.csv files into one dataset."""
    file_pattern = os.path.join(desktop_path, "threads_*.csv")
    all_files = glob.glob(file_pattern)

    print(f"Found {len(all_files)} files to merge:")

    df_list = []
    for filename in all_files:
        # Skip any existing master file
        if "MASTER" in filename:
            continue

        try:
            df = pd.read_csv(filename)

            # Ensure Time column is datetime for sorting
            df["Time"] = pd.to_datetime(df["Time"])

            df_list.append(df)
            print(f" - Added: {os.path.basename(filename)} ({len(df)} rows)")
        except Exception as e:
            print(f" - Skipped {os.path.basename(filename)}: {e}")

    if not df_list:
        print("No files to merge.")
        return None

    master_df = pd.concat(df_list, ignore_index=True)

    # Sort by time and drop duplicates
    master_df = master_df.sort_values(by="Time")
    master_df = master_df.drop_duplicates()

    master_df.to_csv(master_path, index=False)
    print(f"\nSuccess. Saved {len(master_df)} rows to: {master_path}")

    return master_df
```

```
def build_sir_ready(master_df):
    """Create the SIR-ready dataset and save it."""
    # Clean views
    master_df["Views_Cleaned"] = master_df["Views"].apply(clean_views)

    # Ensure Time is datetime
    master_df["Time"] = pd.to_datetime(master_df["Time"])

    # Sort by post and time
    master_df = master_df.sort_values(by=["Post_Label", "Time"])

    # Compute hours elapsed per post
    master_df["Hours_Elapsed"] = master_df.groupby("Post_Label")["Time"].transform
        (
        lambda x: (x - x.min()).dt.total_seconds() / 3600
    )

    # Keep only the columns needed for SIR model
    final_df = master_df[["Post_Label", "Time",
                          "Hours_Elapsed", "Views_Cleaned"]]

    final_df.to_csv(sir_ready_path, index=False)
    print(
        f"SIR-ready file saved with {len(final_df)} rows to: {sir_ready_path}")


def main():
    master_df = merge_to_master()
    if master_df is None:
        return
    build_sir_ready(master_df)


if __name__ == "__main__":
    main()
```

B Python code for SIR fitting

Listing 2: Python script for fitting the SIR model to Threads data

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from scipy.optimize import curve_fit

CSV_FILE = "/Users/kevinlin/Desktop/Threads_Project/threads_SIR_ready.csv"

# Plot only these labels (set to None to plot all)
# Example: PLOT_LABELS = ['trends', 'Clothing', 'Costco']
PLOT_LABELS = None

# Minimum number of points required to fit a curve
MIN_POINTS = 4

# R0 threshold for marking a post as "viral"
R0_THRESHOLD = 1.0
```

```
def sir_derivatives(y, t, beta, gamma):
    """SIR model derivatives."""
    S, I, R = y
    dSdt = -beta * S * I
    dIdt = beta * S * I - gamma * I
    dRdt = gamma * I
    return dSdt, dIdt, dRdt


def make_fit_func(initial_I0):
    """
    Return a function f(t, beta, gamma) giving the cumulative
    infected proportion 1 - S(t) for the SIR model.
    """
    def fit_func(t, beta, gamma):
        S0 = 1.0 - initial_I0
        I0 = initial_I0
        R0_val = 0.0
        y0 = [S0, I0, R0_val]

        sol = odeint(sir_derivatives, y0, t, args=(beta, gamma))
        S_t, _, _ = sol.T
        return 1.0 - S_t

    return fit_func

def load_data(csv_path):
    """Load data from CSV into a pandas DataFrame."""
    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"Could not find file at {csv_path}")
    return pd.read_csv(csv_path)


def fit_post(subset):
    """
    Fit SIR model for a single post (subset of the dataframe).

    Returns:
        beta, gamma, R0, t_data, y_data, t_smooth, y_smooth
    """
    subset = subset.sort_values(by="Hours_Elapsed")
    t_data = subset["Hours_Elapsed"].values
    y_data = subset["Views_Cleaned"].values

    if len(t_data) < MIN_POINTS:
        raise ValueError("Too few data points for fitting")

    # Effective population size used for normalization
    N = y_data.max() * 1.2
    y_norm = y_data / N

    initial_I0 = y_norm[0]
    fit_func = make_fit_func(initial_I0)

    # Fit beta and gamma
    params, _ = curve_fit(
        fit_func,
        t_data,
        y_norm,
        p0=[0.5, 0.1],
        bounds=(0, [10.0, 5.0]),
    )
```

```
    beta, gamma = params

    # Avoid division by very small gamma
    gamma = max(gamma, 1e-4)
    R0 = beta / gamma

    # Smooth curve for plotting
    t_smooth = np.linspace(0, t_data.max() + 12, 200)
    y_smooth_norm = fit_func(t_smooth, beta, gamma)
    y_smooth = y_smooth_norm * N

    return beta, gamma, R0, t_data, y_data, t_smooth, y_smooth

def main():
    df = load_data(CSV_FILE)
    all_labels = df["Post_Label"].unique()

    if PLOT_LABELS is None:
        labels_to_plot = all_labels
    else:
        labels_to_plot = [lbl for lbl in all_labels if lbl in PLOT_LABELS]

    print(f"Found {len(all_labels)} post labels: {all_labels}\n")

    results = []

    print(f"{'Label':<15} | {'Beta':<8} | {'Gamma':<8} | {'R0':<10} | Status")
    print("-" * 70)

    plt.figure(figsize=(12, 7))

    # Use a colormap for consistent but simple colors
    cmap = plt.get_cmap("tab10")

    for idx, label in enumerate(labels_to_plot):
        subset = df[df["Post_Label"] == label].copy()

        try:
            beta, gamma, R0, t_data, y_data, t_smooth, y_smooth = fit_post(
                subset)
            status = "Viral" if R0 > R0_THRESHOLD else "Not viral"

            print(
                f"{label:<15} | "
                f"{beta:.4f}   | "
                f"{gamma:.4f}   | "
                f"{R0:.4f}     | {status}"
            )

            color = cmap(idx % 10)

            # Plot data points
            plt.scatter(
                t_data,
                y_data,
                color=color,
                s=15,
                alpha=0.6,
                marker="o",
            )

            # Plot fitted SIR curve
```

```
            plt.plot(
                t_smooth,
                y_smooth,
                color=color,
                linewidth=2.2,
                label=f"{label} ($R_0={R0:.1f}$)",
            )

            results.append((label, R0))

        except Exception as e:
            print(f"{label:<15} | Fit failed: {e}")

    # Sort legend by R0 in descending order
    if results:
        handles, legend_labels = plt.gca().get_legend_handles_labels()
        R0_map = {lab.split(" ($")[0]: R for lab, R in results}
        order = sorted(
            range(len(legend_labels)),
            key=lambda i: R0_map.get(legend_labels[i].split(" ($")[0], 0.0),
            reverse=True,
        )
        handles = [handles[i] for i in order]
        legend_labels = [legend_labels[i] for i in order]

        plt.legend(
            handles,
            legend_labels,
            bbox_to_anchor=(1.02, 1),
            loc="upper left",
            borderaxespad=0.0,
            fontsize=9,
        )

    plt.title("SIR Model Analysis: Viral Growth on Threads", fontsize=16)
    plt.xlabel("Hours Since First Observation", fontsize=13)
    plt.ylabel("Total Views", fontsize=13)
    plt.grid(alpha=0.3)
    plt.tight_layout()

    save_path = os.path.join(
        os.path.expanduser("~/Desktop"),
        "threads_ALL_analysis.png",
    )
    plt.savefig(save_path, dpi=300)
    print(f"\nAnalysis complete. Graph saved to: {save_path}")
    plt.show()


if __name__ == "__main__":
    main()
```

C Appendix: Mathematica Code for the Nonlinear SIR Model

Bifurcation Diagram in Terms of R_0

The following Mathematica code produces a one–parameter bifurcation diagram for the linearised eigenvalue $\lambda_2(R_0) = \gamma(R_0 - 1)$ and highlights the crossing at $R_0 = 1$.

```
Clear[gamma, R, lambda2, bifPlot];
```

```
gamma = 0.8;
lambda2[R_] := gamma (R - 1);

bifPlot =
  Plot[lambda2[R], {R, 0, 3},
    AxesLabel -> {"R0", "lambda2"},
    PlotLabel -> "Eigenvalue crossing at R0 = 1",
    PlotRange -> All,
    Epilog -> {
      Red, PointSize[0.02], Point[{1, 0}],
      Dashed, Gray, Line[{{1, -gamma}, {1, gamma}}],
      Dashed, Gray, Line[{{0, 0}, {3, 0}}],
      Text["stable", {0.4, -0.4}],
      Text["unstable", {2.2, 0.4}]
    },
    ImageSize -> Large
  ];

Export["nonlinear_diagram_gr1.eps", bifPlot, "EPS"];
```

Phase Plane, Nullclines, and Fixed Points

The next block generates the phase plane for the SIR model with parameters $\beta = 1.5$, $\gamma = 0.8$, together with the nullclines and the disease–free fixed points along the horizontal axis.

```
Clear[beta, gamma, s, i, R0, fS, fI,
      streamPlot, nullclineS, nullclineI, fixedPts, phasePlanePlot];

beta = 1.5;
gamma = 0.8;
R0 = beta/gamma;

fS[s_, i_] := -beta s i;
fI[s_, i_] :=  beta s i - gamma i;

streamPlot =
  StreamPlot[{fS[s, i], fI[s, i]}, {s, 0, 1}, {i, 0, 0.6},
    StreamStyle -> Arrowheads[0.02],
    AxesLabel -> {"s (susceptible fraction)", "i (infected fraction)"},
    PlotRangePadding -> Scaled[0.02],
    ImageSize -> Large
  ];

nullclineS =
  ContourPlot[fS[s, i] == 0, {s, 0, 1}, {i, 0, 0.6},
    ContourStyle -> {Red, Thick},
    Contours -> {0}
  ];

nullclineI =
  ContourPlot[fI[s, i] == 0, {s, 0, 1}, {i, 0, 0.6},
    ContourStyle -> {Blue, Thick},
```

```
    Contours -> {0}
  ];

fixedPts =
  Graphics[{
    Black, PointSize[0.018],
    Table[Point[{sStar, 0}], {sStar, 0, 1, 0.2}],
    Red, PointSize[0.025], Point[{1, 0}]
  }];

phasePlanePlot =
  Show[streamPlot, nullclineS, nullclineI, fixedPts,
    PlotLabel ->
      Row[{"Phase plane, nullclines and fixed points (R0 = ",
        NumberForm[R0, {3, 2}], ")"}],
    PlotLegends -> Placed[{"s' = 0", "i' = 0"}, {0.8, 0.8}]
  ];

Export["nonlinear_diagram_gr2.eps", phasePlanePlot, "EPS"];
```

Phase Plane for the Costco Post

This code uses the fitted parameters for the Costco post ($\beta = 0.38$, $\gamma = 0.18$) and integrates the SIR system from an initial condition close to the disease-free equilibrium. The trajectory is plotted together with the phase-plane streamlines and the epidemic threshold $s = \gamma/\beta$.

```
Clear[beta, gamma, s, i, R0, fS, fI,
      threshold, stream, nullclines, sol, trajectory, phasePlaneCostco];

beta = 0.38;
gamma = 0.18;
R0 = beta/gamma;

fS[s_, i_] := -beta s i;
fI[s_, i_] :=  beta s i - gamma i;

threshold = gamma/beta;

stream =
  StreamPlot[{fS[s, i], fI[s, i]}, {s, 0, 1.05}, {i, 0, 0.6},
    Axes -> True,
    AxesLabel -> {"s (Susceptible)", "i (Infected)"},
    PlotLabel ->
      Style[
        "Phase Plane: Costco Post (R0 = " <>
          ToString[NumberForm[R0, {3, 2}]] <> ")",
        14, Bold
      ],
    StreamStyle -> {Gray, Arrowheads[0.02]},
    StreamPoints -> Fine,
    GridLines -> Automatic
  ];
```

```
nullclines =
  Graphics[{
    {Gray, Dashed, Thickness[0.005],
     Line[{{0, 0}, {1.05, 0}}]},
    {Darker[Green], Dashed, Thickness[0.005],
     Line[{{threshold, 0}, {threshold, 0.6}}]},
    Text[
      Style[
        "Epidemic Threshold\ns = " <>
          ToString[NumberForm[threshold, {3, 2}]],
        10, Darker[Green]
      ],
      {threshold + 0.15, 0.5}
    ]
  }];

sol =
  NDSolve[
    {
      s'[t] == -beta s[t] i[t],
      i'[t] ==  beta s[t] i[t] - gamma i[t],
      s[0] == 0.99,
      i[0] == 0.01
    },
    {s, i}, {t, 0, 100}
  ];

trajectory =
  ParametricPlot[
    Evaluate[{s[t], i[t]} /. sol],
    {t, 0, 100},
    PlotStyle -> {Red, Thickness[0.007]}
  ];

phasePlaneCostco =
  Show[stream, nullclines, trajectory,
    PlotRange -> {{0, 1.05}, {0, 0.6}},
    ImageSize -> Large,
    Frame -> True,
    FrameLabel -> {"Susceptible Fraction (s)", "Infected Fraction (i)"}
  ];

Export["nonlinear_diagram_gr3.eps", phasePlaneCostco, "EPS"];
```